Thetawave.ai

💻 控制反轉與樣板方法設計策略解析

Generated by Thetawave.ai

3.1 景點 - 樣板方法_留同存異的萃取能力.mp4

🧩 分組策略設計與樣板方法(Template Method)模式解析

1. 問題背景與需求說明

- 目標：為水球軟體學院設計「學員分組(Grouping)」機制，依不同策略將學員分至不同組別。
- 學員資料包含：名字(Name)、年資(Experience)、程式語言專長(Language)、職稱(Job Title)、空閒時間段(Available Time Slots)。
- 每組別有一組編號(Number)。
- 需求範例：
  - **Language-Based Grouping Strategy**：以程式語言專長切第一刀，組別最低人數6人切第二刀，最後將人數不足6人的組別合併至相同語言組別中。
  - **Time-Slot-Based Grouping Strategy**：以空閒時間切第一刀，最低6人切第二刀，最後任意合併人數不足的組別。

---

2. 物件導向分析與初版設計

- **類別設計**：
  - Student類別：封裝學員資料。
  - Group類別：包含多個Student。
  - Language-Based Grouping Strategy類別：實作語言基礎分組策略，輸入Students，輸出Groups。

Thetawave.ai

- 初版實作流程：
  - 主方法(Main Method)創建策略物件並讀入學員資料。
  - 呼叫策略方法進行分組，回傳分好組的列表。
  - 印出分組結果。
- 主要技術：
  - 第一刀使用Key-Value雜湊結構依程式語言分類。
  - 第二刀使用Group類別的Split by Size方法，切成多個至少6人的子組。
  - 第三步透過條件篩選合併人數不足的組別。

---

3. 擴充需求與策略模式應用

需求變化	差異說明	合併標準變動
Language-Based Grouping	第一刀依程式語言切分	只合併同語言組別
Time-Slot-Based Grouping	第一刀依學員空閒時間切分	可任意合併人數不足組別
Job Title-Based Grouping	第一刀依職稱切分	只合併同職稱組別

- **策略模式(Strategy Pattern)**：
  - 將分組演算法封裝成多個策略類別。
  - 介面Grouping Strategy定義分組行為。
  - 客戶端可自由切換不同策略。
- 問題：兩個策略類別中有大量重複程式碼，僅少部分行為不同，導致維護困難。

---

4. 面臨的三道設計力道(Forces)

力道編號	力道描述
1	多個類似分組行為導致大量重複程式碼
2	重複程式碼中僅小部分行為會變動

3	新增分組策略需複製貼上修改，擴充困難且維護不易

- **核心問題**：如何在開發新分組策略時，只撰寫變動部分，避免重複程式碼？

---

5. 樣板方法(Template Method)解決方案

> 「樣板方法是一種設計模式，將固定不變的行為封裝成一個模板方法，並將可變部分定義為抽象或掛鉤(hook)方法，讓子類別實作。」

- **抽象類別**Cut-Based Grouping Strategy：
  - 實作Grouping Strategy介面。
  - 實作一個Group()樣板方法，流程包含：
    1. **第一刀切法(Cut-by)** — 抽象方法，依子類別定義如何切分學員。
    2. **第二刀切法(Split by Size)** — 固定邏輯，將大組切成最小6人的子組。
    3. **合併邏輯(Meet-Merge Criteria)** — 預設為任意組合併，子類別可覆寫決定合併標準。
- **樣板方法結構示意**：

```
Group():
  1. 使用 Cut-by() 切分學員
  2. 對每組呼叫 Split by Size(6)
  3. 篩選人數足夠與不足組別
  4. 依 Meet-Merge Criteria 合併不足組別
  5. 回傳最終組別列表
```

- **抽象方法與掛鉤(hook)方法**：

方法名稱	功能說明	實作方式

Cut-by(Student)	決定如何以學員屬性切分第一刀	抽象方法，子類別實作
Meet-Merge Criteria(Group a, Group b)	判斷兩組是否可合併	具體方法，可被覆寫

Metawave.ai

- **方法權限**：protected，允許子類別存取及覆寫。

---

6. 具體子類別實作

子類別名稱	第一刀切法屬性	合併條件說明
Language-Based Grouping Strategy	使用學員程式語言專長	只合併相同程式語言的組別
Time-Slot-Based Grouping Strategy	使用學員空閒時間	可任意合併人數不足的組別
Job Title-Based Grouping Strategy	使用學員職稱	只合併相同職稱的組別

- 子類別只需覆寫Cut-by和Meet-Merge Criteria兩方法，其他共通邏輯由抽象類別管理。
- 減少複製貼上程式碼，提升擴充性與維護性。

---

7. 角色與責任總結

類別/介面	主要責任
Grouping Strategy介面	定義分組策略的行為介面，供多種策略實作
Cut-Based Grouping Strategy	抽象類別，實作固定流程(樣板方法)，定義可變動部分為抽象方法或掛鉤
各子類別(語言、時間、職稱)	實作切分依據(Cut-by)、合併條件(Meet-Merge Criteria)
Group	封裝組別成員，提供切分子組(Split by Size)等功能
Student	封裝學員資料屬性

---

8. 設計模式關聯與應用說明

Thetawave.ai

設計模式名稱	角色	功能說明
策略模式(Strategy Pattern)	封裝多個分組演算法，介面化策略切換	讓系統可在執行時動態切換不同分組策略，解決行為變動性
樣板方法(Template Method)	抽象類別定義固定流程，子類別覆寫變動步驟	提取重複程式碼，將可變部分留給子類別實作，提升程式碼重用與擴充性

- 先以策略模式管理不同分組演算法，解決行為變動性問題。
- 再用樣板方法將策略內部重複程式碼抽象化，解決重複程式碼與維護問題。
- 兩者結合實現既靈活又具維護性的分組機制架構。

---

9. 重要觀念與容易混淆處說明

觀念	說明	容易混淆點與澄清
樣板方法 (Template Method)	是一個方法內部的行為模板，定義固定流程與可變步驟	並非整個類別模式，是方法層級的設計技巧
掛鉤 (Hook)	具體且有預設實作的方法，子類別可選擇覆寫	不是抽象方法，子類別可自主決定是否覆寫
行為變動性 (Behavioral Variation)	程式碼中少部分會依策略不同而變動的行為	與重複程式碼區分，變動部分應抽象成方法，避免複製貼上
策略模式與樣板方法差異	策略模式封裝整個行為策略，樣板方法抽象固定流程內的部分行為	策略用來切換整體演算法，樣板方法用於演算法內部結構優化

---

10. 實作流程與步驟整理

步驟	說明
1. 定義Grouping Strategy介面	規範分組方法，供不同策略實作
2. 實作Cut-Based Grouping Strategy抽象類別	將共通的分組流程寫成樣板方法，抽象切分與合併條件方法

3. 實作具體子類別覆寫變動方法	如Language-Based、Time-Slot-Based、Job Title-Based 分別覆寫Cut-by與Meet-Merge Criteria
4. 主程式創建策略物件並執行分組	根據需求選擇策略後呼叫Group()取得分組結果
5. 測試與驗證分組結果	確認分組符合預期條件，例如組別人數、合併規則等

---

11. 範例程式碼示意（重點方法）

```
abstract class CutBasedGroupingStrategy implements GroupingStrategy {
  public List<Group> group(List<Student> students) {
    Map<String, Group> initialGroups = new HashMap<>();
    // 第一刀切法：依Cut-by決定Key分組
    for (Student s : students) {
      String key = cutBy(s);
      initialGroups.computeIfAbsent(key, k -> new Group()).addStudent(s);
    }
    // 第二刀切法：依大小分割子組
    List<Group> splitGroups = new ArrayList<>();
    for (Group g : initialGroups.values()) {
      splitGroups.addAll(g.splitBySize(6));
    }
    // 合併不足組別
    List<Group> fullGroups = new ArrayList<>();
    List<Group> smallGroups = new ArrayList<>();
    for (Group g : splitGroups) {
      if (g.size() >= 6) fullGroups.add(g);
      else smallGroups.add(g);
    }
    // 合併邏輯
    for (Group small : smallGroups) {
      for (Group full : fullGroups) {
        if (meetMergeCriteria(small, full)) {
          full.merge(small);
          break;
        }
      }
    }
    return fullGroups;
  }

  protected abstract String cutBy(Student s);
```

```
  protected boolean meetMergeCriteria(Group a, Group b) {
    return true; // 預設任意合併
  }
}

class LanguageBasedGroupingStrategy extends CutBasedGroupingStrategy {
  @Override
  protected String cutBy(Student s) {
    return s.getLanguage();
  }
  @Override
  protected boolean meetMergeCriteria(Group a, Group b) {
    return a.getKey().equals(b.getKey()); // 只合併同語言組別
  }
}

class TimeSlotBasedGroupingStrategy extends CutBasedGroupingStrategy {
  @Override
  protected String cutBy(Student s) {
    return s.getAvailableTimeSlot();
  }
  // 使用預設合併邏輯
}
```

Thetawave.ai

---

12. 重要結論判斷

樣板方法透過將重複的分組流程封裝在抽象類別中，並將變動的切分條件和合併標準設為子類別實作的抽象或掛鉤方法，將共通邏輯與變動行為分離，極大地提升了代碼的重用性與擴充性。這使得新增類似分組策略時，開發者僅需聚焦於少量變動部分的實作，避免冗長複製貼上，且保持系統架構清晰，利於長期維護與演進。

3.2 用樣板方法來實現[控制反轉].mp4

樣板方法與控制反轉的關係 🎭

> **控制反轉（Inversion of Control, IoC）**是將流程的控制權從應用程式轉移到框架，使開發者能專注於業務邏輯，實現**關注點分離**。

Thetawave.ai

1. 專有名詞層級的區分

名詞	性質	說明
樣板方法	設計模式	用來去除重複程式碼的進階手段。
依賴反轉	設計原則	關注如何反轉傳統依賴方向，提升系統模組間的鬆耦合。
控制反轉	設計原則	把流程控制權從應用程式交給框架，促進關注點分離。
依賴反轉之重構三步驟	重構模式	透過三步驟重構來實現依賴反轉設計原則。

> 理解這些名詞時，要注意它們聚焦的層級不同，例如控制反轉和依賴反轉都是設計原則，而重構三步驟是實作依賴反轉的具體方法。

控制反轉的定義、目的與實現方法

項目	內容
定義	把流程的控制權從應用程式轉移到框架。
目的	實現**關注點分離**，讓開發者能專注在業務邏輯，而非繁瑣的底層流程控制。
實現方法	使用已學過的**樣板方法**，透過三步驟抽象流程並交由框架管理。

用藝人與經紀人比喻理解控制反轉 🎤🎬

- **沒有經紀人時：**藝人必須自己管理所有行程、接洽和繁瑣事務，流程控制權全在藝人手中，容易分心且效率低。
- **有經紀人時：**經紀人接管流程控制，安排行程與資源，藝人只需專注於表現，實現關注點分離，雙方皆能提升表現。

> 這比喻說明控制反轉就是讓框架（經紀人）掌控流程，應用程式（藝人）專注核心業務邏輯。

開發流程中控制反轉的實際運用

無框架時的流程控制

- 應用程式需自行實作啟動、運行、結束三大生命週期。
- 例如命令行程式需寫 while 迴圈持續接收用戶輸入，控制整個流程。
- 重複且繁瑣的流程控制程式碼易導致失焦，且每個專案都需重複撰寫。

有框架時的流程控制

- 流程控制權移交給框架，應用程式只需關注變動的業務邏輯。
- 框架負責啟動流程、迴圈控制、結束流程等重複部分。
- 應用程式透過覆寫抽象方法（掛鉤）來實現特定業務邏輯。

使用樣板方法實現控制反轉的三大步驟

步驟	說明
1. 區分重複與變動	識別流程中哪些部分是重複固定的，哪些是會變化的業務邏輯。
2. 流同存異	將重複部分（流同）抽取為樣板方法，變異部分（存異）設計為抽象步驟或掛鉤，允許多形（多態）覆寫。
3. 定義框架生命週期	設計穩定且合理的生命週期，決定應用程式可獲得控制權的時機點，平衡框架控制與應用彈性。

Thetawave.ai

控制反轉流程示意

框架（經紀人）控制流程：
啟動 → 啟動後掛鉤 → 主要迴圈（呼叫應用程式業務邏輯） → 結束前掛鉤 → 結束

- 啟動、主要迴圈與結束過程由框架負責。
- 應用程式僅在掛鉤點被呼叫，執行覆寫後的業務邏輯。
- 控制權由框架完全掌握，流程透明且穩定。

框架生命週期掛鉤範例

掛鉤名稱	作用說明	類型
啟動後處理	程式啟動完成後，應用程式可執行初始化	抽象方法或預設實作掛鉤
業務邏輯	主要運行時，應用程式執行具體功能	抽象方法
結束前處理	程式結束前，應用程式可釋放資源等	抽象方法或預設實作掛鉤

> 透過這些掛鉤，框架提供穩定的生命週期結構，讓應用程式在特定時機點被呼叫，完成業務邏輯。

控制反轉與依賴注入的關係

- **依賴注入（Dependency Injection, DI）**是將執行時所需的依賴對象注入至應用程式的技術。
- 在控制反轉框架中，框架不僅掌控流程，還會在適當時機注入所需依賴，幫助應用程式減少對細節的關注。

- 依賴注入類似經紀人幫藝人安排造型師、化妝師等資源，讓藝人專注表現。

TheNewWave.ai

簡化類別圖示例

類別	角色說明
抽象類別（框架）	定義樣板方法（啟動、運行迴圈、結束）及抽象掛鉤方法。
應用程式子類	覆寫框架抽象掛鉤，實作具體業務邏輯。

設計框架生命週期的思考重點

- 框架需決定開放給應用程式多少流程控制權，避免過度開放導致複雜與不穩定。
- 框架生命週期的定義需兼顧多種應用程式需求，保持彈性與穩定性。
- 生命週期設計適當，能確保框架與應用程式間的雙贏合作。

控制反轉的實務意涵

控制反轉將流程控制權從應用程式轉移至框架，強化了**關注點分離**，使得開發者能夠專注於業務邏輯的開發，而框架負責底層流程管理與資源注入；這種分工不僅提升了程式碼的可維護性與可重用性，也促進了系統的整體穩定性與擴展性。